Crumb & Co: Plan Analyzer Code

Python model that rebuilds an AI-drafted business plan's numbers, checks the AI's claims, and flags weak assumptions. Same code as the file plan_analyzer.py.

How to run it

1. Save the code below as `plan_analyzer.py`. It needs only the Python standard library (Python 3.8 or newer).
2. Run `python plan_analyzer.py` to analyse the built-in Crumb & Co data.
3. Run `python plan_analyzer.py --out report.md` to also save a Markdown report.
4. Run `python plan_analyzer.py --dump-config plan.json` to write the inputs to a file, edit them, then run `python plan_analyzer.py --config plan.json`.

The script exits with code 1 when any check fails, so you can use it in a script or a checklist.

What it checks

Check	What it does	Error-log ID
Arithmetic	Recomputes average order value, contribution, break-even, the 6-month projection, payback and steady-state surplus, and compares them with the AI's stated figures	
Fixed-cost base	Flags a plan that uses different fixed costs in different sections	E8, E9
Wastage rounding	Notes where the stated wastage column differs from 8% of revenue	E10
Prices	Compares each price with competitor listings you supply	E4
Cost ratios	Warns when every product's cost ratio is suspiciously uniform	E5
Packaging	Compares packaging per unit with the parts you could price	E6
Licensing	Compares the licensing budget line with the actual fee	E2, E3

Check	What it does	Error-log ID
Gas and electricity	Compares the budget with a commercial cylinder and electric baking	E7
Missing costs	Lists costs the draft leaves out, with their effect	E12
Capacity	Tests whether the order ramp fits your available hours (needs your inputs)	E13
Sensitivity	Shows how surplus and break-even move with prices, costs, delivery and order volume	

The code

```
#!/usr/bin/env python3
"""
plan_analyzer.py: a model that analyses an AI-drafted small-business plan.

What it does
  1. Rebuilds the plan's financial model from the raw inputs (unit economics,
     12-month projection, break-even, payback, sensitivity).
  2. Compares the AI's stated figures ("claims") with the recomputed ones.
  3. Runs reality checks against benchmark data you supply (competitor
prices,
     fees, utility costs, packaging parts).
  4. Flags patterns that suggest weak assumptions (uniform cost ratios,
     inconsistent fixed-cost bases, missing cost categories).
  5. Prints a report and can save it as Markdown.

Default data is the Crumb & Co (Chennai home bakery) draft. Benchmarks are
the references found on 24 October 2026 and are mostly undated listings, so
confirm them before relying on them.

Usage
  python plan_analyzer.py                      # run with the built-in data
  python plan_analyzer.py --out report.md      # also save a Markdown report
  python plan_analyzer.py --config my_plan.json # override any input
  python plan_analyzer.py --dump-config plan.json   # write the defaults to
edit

Only the Python standard library is needed (Python 3.8+).
"""
import argparse
import copy
```

```
import json
import math
import statistics
import sys

# ---------------------------------------------------------------------------
# 1. INPUTS: the AI draft's raw assumptions, its stated claims, and
benchmarks
# ---------------------------------------------------------------------------
DEFAULT_PLAN = {
    "business": "Crumb & Co (home bakery, Chennai)",
    "weeks_per_month": 4.33,
    "wastage_pct": 0.08,              # share of revenue lost to wastage and
discounts
    "delivery_per_order": 0.0,        # AI draft: customer pays delivery
    "payment_fee_pct": 0.0,           # AI draft: UPI assumed free
    "products": [
        # name, price, ingredients, packaging, steady-state orders per week
        {"name": "Eggless cake, 1/2 kg", "price": 750, "ingredients": 230,
"packaging": 60, "orders_per_week": 8},
        {"name": "Eggless cake, 1 kg", "price": 1400, "ingredients": 430,
"packaging": 90, "orders_per_week": 3},
        {"name": "Brownie box (6)", "price": 480, "ingredients": 150,
"packaging": 35, "orders_per_week": 9},
        {"name": "Cookie box (10)", "price": 400, "ingredients": 120,
"packaging": 30, "orders_per_week": 7},
        {"name": "Gift hamper", "price": 900, "ingredients": 270,
"packaging": 80, "orders_per_week": 3},
    ],
    "startup": {
        "Equipment": 35000,
        "Licensing and packaging": 20000,
        "Opening ingredient stock": 15000,
        "Marketing and photography": 15000,
        "Cash reserve": 15000,
    },
    "fixed_monthly": {
        "Extra electricity and gas": 2500,
        "Ongoing marketing": 1500,
        "Phone, internet and apps": 500,
        "Equipment upkeep": 500,
        "Cleaning and hygiene": 500,
        "FSSAI, insurance and misc": 300,
    },
    "marketing_key": "Ongoing marketing",
    "owner_pay_scenarios": [0, 15000, 25000],
```

```
    "base_owner_pay": 15000,
    "ramp_orders_per_week": [9, 14, 18, 20, 23, 28],
    "steady_state_orders_per_week": 30,
    "uses_gas": None,                  # True, False or None (unknown)
    # Optional: minutes of work per unit, for the capacity check. None skips
it.
    "minutes_per_unit": None,          # e.g. {"Eggless cake, 1/2 kg": 90,
...}
    "available_hours_per_week": None,  # e.g. 25
    # The AI's own stated figures, to be checked against the recomputed ones.
    "claims": {
        "startup_total": 100000,
        "aov": 667,
        "ingredients_packaging_per_order": 255,
        "wastage_per_order": 53,
        "contribution_per_order": 359,
        "fixed_with_marketing": 5800,
        "fixed_used_in_projection": 4300,
        "fixed_used_in_break_even": 5800,
        "break_even_orders_month": {"0": 17, "15000": 58, "25000": 86},
        "break_even_orders_week": {"0": 4, "15000": 14, "25000": 20},
        "break_even_revenue": {"0": 11300, "15000": 38700, "25000": 57400},
        "revenue": [26000, 40700, 52000, 58000, 66700, 80700],
        "ingredients_packaging": [9900, 15600, 19900, 22200, 25500, 30900],
        "wastage": [2100, 3200, 4100, 4600, 5300, 6400],
        "contribution": [14000, 21900, 28000, 31200, 35900, 43400],
        "surplus_before_pay": [9700, 17600, 23700, 26900, 31600, 39100],
        "six_month_surplus_before_pay": 148600,
        "payback_month_at_base_pay": 8,
        "steady_surplus_before_pay": {"20": 25300, "25": 33100, "30": 40800},
    },
}

# Benchmarks gathered from outside the draft. Replace with your own evidence.
DEFAULT_BENCHMARKS = {
    "as_of": "2026-10-24",
    "competitor_prices": {          # Chennai listings found (undated)
        "Eggless cake, 1/2 kg": [530, 549],
        "Eggless cake, 1 kg": [930, 940, 999],
        "Brownie box (6)": [375],
    },
    "licence_fee_per_year": 100,    # FSSAI Basic Registration fee (confirm
on FoSCoS)
    "commercial_lpg_cylinder": 2984,  # 19 kg, Chennai, from 1 Oct 2026
    "electric_oven_monthly_range": [900, 1500],  # my illustrative estimate
    "packaging_parts_range": {      # sum of priced parts per unit (low,
```

```
high)
        "Eggless cake, 1/2 kg": [24, 39],
        "Eggless cake, 1 kg": [32, 52],
        "Brownie box (6)": [9, 10],
        "Cookie box (10)": [11, 25],
        "Gift hamper": [20, 27],
    },
    "delivery_per_5km": 50,         # bike delivery, lowest listed rate
}

THRESHOLDS = {
    "price_premium_warn": 0.15,     # price above the highest listing by more
than this: WARN
    "price_premium_high": 0.35,     # ...and by more than this: FAIL
    "cost_ratio_range_warn": 0.05,  # product cost ratios within this spread:
WARN (possibly back-solved)
    "packaging_ratio_warn": 1.5,    # model packaging / top of priced parts
    "packaging_ratio_high": 3.0,
    "money_tolerance": 100,         # rupees, claims rounded to the nearest
hundred
    "licence_line_multiple": 20,    # licensing line vs actual fee
}


# ---------------------------------------------------------------------------
# 2. THE FINANCIAL MODEL
# ---------------------------------------------------------------------------
def round_to(value, step=100):
    return int(round(value / step) * step)


def unit_economics(plan, price_factor=1.0, cost_factor=1.0, delivery=None):
    """Weighted per-order economics at the steady-state product mix."""
    products = plan["products"]
    total_orders = sum(p["orders_per_week"] for p in products)
    delivery = plan["delivery_per_order"] if delivery is None else delivery
    aov = sum(p["price"] * price_factor * p["orders_per_week"] for p in
products) / total_orders
    ing = sum((p["ingredients"] + p["packaging"]) * cost_factor *
p["orders_per_week"]
              for p in products) / total_orders
    wastage = aov * plan["wastage_pct"]
    fees = aov * plan["payment_fee_pct"]
    contribution = aov - ing - wastage - delivery - fees
    return {"aov": aov, "ingredients_packaging": ing, "wastage": wastage,
            "delivery": delivery, "fees": fees, "contribution": contribution,
```

```
            "variable_cost": ing + wastage + delivery + fees,
            "total_orders_per_week": total_orders}


def fixed_total(plan, include_marketing=True):
    total = sum(plan["fixed_monthly"].values())
    if not include_marketing:
        total -= plan["fixed_monthly"].get(plan["marketing_key"], 0)
    return total


def project_months(plan, orders_per_week, fixed, owner_pay, ue=None):
    """Month-by-month projection, using whole orders and a rounded average
order
    value the way the draft does, so figures are comparable like for like."""
    ue = ue or unit_economics(plan)
    aov = round(ue["aov"])
    rows = []
    for wk in orders_per_week:
        orders = round(wk * plan["weeks_per_month"])
        revenue = orders * aov
        ing = orders * round(ue["ingredients_packaging"])
        wastage = revenue * plan["wastage_pct"]
        other = orders * ue["delivery"] + revenue * plan["payment_fee_pct"]
        contribution = revenue - ing - wastage - other
        before = round_to(contribution) - fixed
        rows.append({"orders": orders, "revenue": revenue,
"ingredients_packaging": ing,
                     "wastage": wastage, "contribution": contribution,
                     "surplus_before_pay": before, "surplus_after_pay":
before - owner_pay})
    return rows


def break_even(plan, fixed, owner_pay, ue=None):
    ue = ue or unit_economics(plan)
    if ue["contribution"] <= 0:
        return None
    orders_month = math.ceil((fixed + owner_pay) / ue["contribution"])
    orders_week = math.ceil(orders_month / plan["weeks_per_month"])
    revenue = orders_month * round(ue["aov"])
    return {"orders_month": orders_month, "orders_week": orders_week,
"revenue": revenue}


def payback_month(rows, startup_total):
```

```
    """Payback the way the draft states it: cumulative surplus after pay,
then the
    last month's run rate carried forward until the startup cost is
recovered."""
    cumulative = 0
    for i, r in enumerate(rows, start=1):
        cumulative += r["surplus_after_pay"]
        if cumulative >= startup_total:
            return i
    last = rows[-1]["surplus_after_pay"]
    if last <= 0:
        return None
    return len(rows) + math.ceil((startup_total - cumulative) / last)


def steady_surplus(plan, orders_per_week, fixed, ue=None):
    ue = ue or unit_economics(plan)
    return orders_per_week * plan["weeks_per_month"] * ue["contribution"] -
fixed


# ---------------------------------------------------------------------------
# 3. FINDINGS
# ---------------------------------------------------------------------------
class Finding:
    LEVELS = ("PASS", "INFO", "WARN", "FAIL")

    def __init__(self, level, area, message, ref=""):
        assert level in self.LEVELS
        self.level, self.area, self.message, self.ref = level, area, message,
ref

    def line(self):
        ref = f" (log: {self.ref})" if self.ref else ""
        return f"[{self.level}] {self.area}: {self.message}{ref}"


def check_close(findings, area, claimed, computed, tol, ref="", unit=""):
    diff = abs(claimed - computed)
    if diff <= tol:
        findings.append(Finding("PASS", area, f"claimed {claimed:,.0f}{unit}
matches recomputed {computed:,.0f}{unit}"))
    else:
        findings.append(Finding("FAIL", area, f"claimed {claimed:,.0f}{unit}
but recomputed {computed:,.0f}{unit} (difference {diff:,.0f})", ref))
```

```
def arithmetic_checks(plan):
    f, c = [], plan["claims"]
    ue = unit_economics(plan)
    tol = THRESHOLDS["money_tolerance"]
    fixed_all = fixed_total(plan, True)
    fixed_no_mkt = fixed_total(plan, False)

    check_close(f, "Startup total", c["startup_total"],
sum(plan["startup"].values()), 0)
    check_close(f, "Average order value", c["aov"], ue["aov"], 1)
    check_close(f, "Ingredients and packaging per order",
c["ingredients_packaging_per_order"], ue["ingredients_packaging"], 1)
    check_close(f, "Wastage per order", c["wastage_per_order"],
ue["wastage"], 1)
    check_close(f, "Contribution per order", c["contribution_per_order"],
ue["contribution"], 1)
    check_close(f, "Fixed costs including marketing",
c["fixed_with_marketing"], fixed_all, 0)

    # Consistency: the draft used two different fixed-cost bases.
    used_p, used_b = c["fixed_used_in_projection"],
c["fixed_used_in_break_even"]
    if used_p != used_b:
        f.append(Finding("FAIL", "Fixed-cost base",
                         f"projection uses {used_p:,} a month but break-even
uses {used_b:,}. "
                         f"The gap is {used_b - used_p:,}, which equals
ongoing marketing "
                         f"({plan['fixed_monthly'].get(plan['marketing_key'],
0):,}).", "E8, E9"))
    else:
        f.append(Finding("PASS", "Fixed-cost base", "projection and break-
even use the same fixed costs"))

    # Break-even by owner's-pay scenario
    for pay in plan["owner_pay_scenarios"]:
        be = break_even(plan, used_b, pay, ue)
        key = str(pay)
        if be is None:
            f.append(Finding("FAIL", f"Break-even (pay {pay:,})",
"contribution per order is not positive"))
            continue
        check_close(f, f"Break-even orders a month (pay {pay:,})",
c["break_even_orders_month"][key], be["orders_month"], 0)
        check_close(f, f"Break-even orders a week (pay {pay:,})",
```

```
c["break_even_orders_week"][key], be["orders_week"], 0)
        check_close(f, f"Break-even revenue (pay {pay:,})",
c["break_even_revenue"][key], be["revenue"], tol)

    # 6-month projection, compared like for like with the draft's rounding
    rows = project_months(plan, plan["ramp_orders_per_week"], used_p,
plan["base_owner_pay"], ue)
    for name, claimed_key, computed_key in (
            ("Revenue", "revenue", "revenue"),
            ("Ingredients and packaging", "ingredients_packaging",
"ingredients_packaging"),
            ("Contribution", "contribution", "contribution"),
            ("Surplus before owner's pay", "surplus_before_pay",
"surplus_before_pay")):
        for i, row in enumerate(rows):
            computed = row[computed_key]
            claimed = c[claimed_key][i]
            if name == "Contribution":
                computed = round_to(computed)
            if abs(claimed - round_to(computed)) > tol:
                f.append(Finding("FAIL", f"{name}, month {i + 1}",
                                 f"claimed {claimed:,} but recomputed
{round_to(computed):,}"))
    # Wastage column: nearest-100 rounding differences are reported, not
failed
    off = []
    for i, row in enumerate(rows):
        nearest = round_to(row["wastage"])
        if abs(c["wastage"][i] - nearest) > tol:
            f.append(Finding("FAIL", f"Wastage, month {i + 1}", f"claimed
{c['wastage'][i]:,} but recomputed {nearest:,}"))
        elif c["wastage"][i] != nearest:
            off.append((i + 1, c["wastage"][i], row["wastage"]))
    if off:
        detail = "; ".join(f"month {m}: stated {s:,} vs 8% = {w:,.0f}" for m,
s, w in off)
        f.append(Finding("INFO", "Wastage column rounding",
                         f"differs from nearest-100 rounding by 100 in
{len(off)} months ({detail}). "
                         "The contribution column still adds up, so this
looks like rounding made to "
                         "keep columns summing.", "E10"))
    total = sum(r["surplus_before_pay"] for r in rows)
    check_close(f, "Six-month surplus before owner's pay",
c["six_month_surplus_before_pay"], total, 0)
```

```
    pb = payback_month(rows, sum(plan["startup"].values()))
    if pb == c["payback_month_at_base_pay"]:
        f.append(Finding("PASS", "Payback month", f"claimed month {pb}
matches recomputed month {pb}"))
    else:
        f.append(Finding("FAIL", "Payback month", f"claimed month
{c['payback_month_at_base_pay']} but recomputed {pb}"))

    for o, claimed in c["steady_surplus_before_pay"].items():
        computed = steady_surplus(plan, int(o), used_b, ue)
        check_close(f, f"Steady-state surplus at {o} orders a week", claimed,
round_to(computed), tol)
    return f


def reality_checks(plan, bench):
    f, th = [], THRESHOLDS
    ue = unit_economics(plan)
    products = {p["name"]: p for p in plan["products"]}

    # Prices against competitor listings
    for name, listings in bench["competitor_prices"].items():
        p = products.get(name)
        if not p:
            continue
        top, low = max(listings), min(listings)
        premium = p["price"] / top - 1
        premium_low = p["price"] / low - 1
        msg = (f"price {p['price']:,} is {premium:.0%} above the highest
listing ({top:,}) "
               f"and {premium_low:.0%} above the lowest ({low:,});
{len(listings)} listing(s) found")
        if premium >= th["price_premium_high"]:
            f.append(Finding("FAIL", f"Price, {name}", msg + ". Check 8 to 10
comparable local sellers.", "E4"))
        elif premium >= th["price_premium_warn"]:
            f.append(Finding("WARN", f"Price, {name}", msg + ".", "E4"))
        else:
            f.append(Finding("PASS", f"Price, {name}", msg + "."))

    # Suspiciously uniform cost ratios
    ratios = {p["name"]: (p["ingredients"] + p["packaging"]) / p["price"] for
p in plan["products"]}
    spread = max(ratios.values()) - min(ratios.values())
    detail = ", ".join(f"{k}: {v:.1%}" for k, v in ratios.items())
    if spread <= th["cost_ratio_range_warn"] and len(ratios) >= 4:
```

```
        f.append(Finding("WARN", "Cost ratios",
                         f"all products sit within {spread:.1%} of each other
({detail}). Real recipes "
                         "rarely cost this uniformly, so the costs may have
been back-solved from a "
                         "target percentage. Cost one product from its
recipe.", "E5"))
    else:
        f.append(Finding("PASS", "Cost ratios", f"ratios vary naturally
({detail})"))

    # Packaging against the parts that could be priced
    for name, (lo, hi) in bench["packaging_parts_range"].items():
        p = products.get(name)
        if not p:
            continue
        ratio = p["packaging"] / hi
        msg = f"model packaging {p['packaging']} vs priced parts {lo} to {hi}
({ratio:.1f} times the top of the range)"
        if ratio >= th["packaging_ratio_high"]:
            f.append(Finding("WARN", f"Packaging, {name}", msg + ". Itemise
stickers, cards, fillers.", "E6"))
        elif ratio >= th["packaging_ratio_warn"]:
            f.append(Finding("INFO", f"Packaging, {name}", msg + "."))
        else:
            f.append(Finding("PASS", f"Packaging, {name}", msg + "."))

    # Licensing line
    lic = plan["startup"].get("Licensing and packaging", 0)
    fee = bench["licence_fee_per_year"]
    if fee and lic >= fee * th["licence_line_multiple"]:
        f.append(Finding("WARN", "Startup: licensing and packaging",
                         f"line is {lic:,} but the registration fee is about
{fee:,} a year. "
                         "Most of the line is unitemised packaging.", "E2"))
    fixed_lic = plan["fixed_monthly"].get("FSSAI, insurance and misc")
    if fixed_lic and fixed_lic * 12 > fee * 10:
        f.append(Finding("INFO", "Fixed: FSSAI, insurance and misc",
                         f"{fixed_lic:,} a month is {fixed_lic * 12:,} a year
against a licence fee of about "
                         f"{fee:,}. Name the rest (for example insurance) or
remove it.", "E3"))

    # Gas and electricity budget
    budget = plan["fixed_monthly"].get("Extra electricity and gas", 0)
    cyl = bench["commercial_lpg_cylinder"]
```

```
    lo, hi = bench["electric_oven_monthly_range"]
    if plan["uses_gas"] is True and budget < cyl:
        f.append(Finding("FAIL", "Gas and electricity",
                         f"budget {budget:,} is below one commercial cylinder
({cyl:,}).", "E7"))
    elif plan["uses_gas"] is False:
        verdict = "covers" if budget >= hi else "may not cover"
        f.append(Finding("PASS" if budget >= hi else "WARN", "Gas and
electricity",
                         f"budget {budget:,} {verdict} electric baking
(estimated {lo:,} to {hi:,}).", "E7"))
    else:
        f.append(Finding("WARN", "Gas and electricity",
                         f"budget {budget:,}: if you bake with gas, one
commercial cylinder ({cyl:,}) "
                         f"exceeds it. If you bake electrically, about {lo:,}
to {hi:,} is expected. "
                         "Set uses_gas to true or false. Domestic cylinders
are not meant for business use.", "E7"))

    # Cost categories the draft leaves out
    missing = []
    if plan["delivery_per_order"] == 0:
        missing.append(f"delivery (about {bench['delivery_per_5km']} per 5 km
if you pay it)")
    if plan["payment_fee_pct"] == 0:
        missing.append("payment fees (0.4% applies only to UPI payments above
2,000)")
    if plan["base_owner_pay"] == 0:
        missing.append("owner's pay")
    if missing:
        d = bench["delivery_per_5km"]
        with_d = unit_economics(plan, delivery=d)["contribution"]
        f.append(Finding("INFO", "Costs not modelled",
                         "; ".join(missing) + f". Paying {d} delivery would
cut contribution per order "
                         f"from {ue['contribution']:,.0f} to {with_d:,.0f}.",
"E12"))

    # Bundled wastage
    f.append(Finding("INFO", "Wastage and discounts",
                     f"{plan['wastage_pct']:.0%} of revenue combines failed
bakes and discounts. "
                     "Track them separately for a month.", "E14"))

    # Capacity check (needs your own time estimates)
```

```
    mpu, hours = plan.get("minutes_per_unit"),
plan.get("available_hours_per_week")
    if mpu and hours:
        need = sum(p["orders_per_week"] * mpu.get(p["name"], 0) for p in
plan["products"]) / 60
        ratio = need / hours
        lvl = "FAIL" if ratio > 1 else "WARN" if ratio > 0.85 else "PASS"
        f.append(Finding(lvl, "Capacity",
                         f"steady-state orders need about {need:.1f} hours a
week against {hours} available "
                         f"({ratio:.0%} of capacity).", "E13"))
    else:
        f.append(Finding("INFO", "Capacity", "skipped. Add minutes_per_unit
and available_hours_per_week "
                                             "to test whether the order ramp
is physically possible.", "E13"))
    return f


# ---------------------------------------------------------------------------
# 4. SENSITIVITY AND REPORT
# ---------------------------------------------------------------------------
def sensitivity(plan):
    scenarios = [
        ("Base case (draft assumptions)", {}),
        ("Prices 10% lower", {"price_factor": 0.90}),
        ("Prices 20% lower", {"price_factor": 0.80}),
        ("Ingredient and packaging costs 15% higher", {"cost_factor": 1.15}),
        ("Delivery of 50 per order paid by you", {"delivery": 50}),
        ("Prices 10% lower and costs 15% higher", {"price_factor": 0.90,
"cost_factor": 1.15}),
    ]
    fixed = fixed_total(plan, True)
    pay = plan["base_owner_pay"]
    out = []
    for label, kw in scenarios:
        ue = unit_economics(plan, **kw)
        surplus = steady_surplus(plan, plan["steady_state_orders_per_week"],
fixed, ue)
        be = break_even(plan, fixed, pay, ue)
        out.append({"scenario": label, "contribution": ue["contribution"],
                    "surplus_before_pay": surplus, "surplus_after_pay":
surplus - pay,
                    "break_even_orders_month": be["orders_month"] if be else
None})
    # Orders sensitivity
```

```
    for wk in (15, 20, 25, plan["steady_state_orders_per_week"]):
        ue = unit_economics(plan)
        s = steady_surplus(plan, wk, fixed, ue)
        out.append({"scenario": f"{wk} orders a week (draft prices)",
"contribution": ue["contribution"],
                    "surplus_before_pay": s, "surplus_after_pay": s - pay,
                    "break_even_orders_month": break_even(plan, fixed, pay,
ue)["orders_month"]})
    return out


def build_report(plan, bench):
    ue = unit_economics(plan)
    fixed = fixed_total(plan, True)
    findings = arithmetic_checks(plan) + reality_checks(plan, bench)
    counts = {lvl: sum(1 for x in findings if x.level == lvl) for lvl in
Finding.LEVELS}
    lines = [f"# Plan analysis: {plan['business']}", "",
             f"Benchmarks as of {bench['as_of']}. Many are undated listings,
so confirm them.", "",
             "## Recomputed unit economics", "",
             f"- Average order value: {ue['aov']:,.2f}",
             f"- Ingredients and packaging per order:
{ue['ingredients_packaging']:,.2f}",
             f"- Wastage and discounts per order: {ue['wastage']:,.2f}",
             f"- Delivery and payment fees per order: {ue['delivery'] +
ue['fees']:,.2f}",
             f"- Contribution per order: {ue['contribution']:,.2f}",
             f"- Monthly fixed costs (with marketing): {fixed:,}",
             f"- Startup total: {sum(plan['startup'].values()):,}", "",
             "## Summary of checks", "",
             "| Level | Count |", "|---|---|"]
    lines += [f"| {lvl} | {counts[lvl]} |" for lvl in Finding.LEVELS]
    for lvl, title in (("FAIL", "Problems"), ("WARN", "Warnings"), ("INFO",
"Notes"), ("PASS", "Passed checks")):
        items = [x for x in findings if x.level == lvl]
        if items:
            lines += ["", f"## {title}", ""] + [f"- {x.line()}" for x in
items]
    lines += ["", "## Sensitivity at steady state", "",
              f"Monthly figures at {plan['steady_state_orders_per_week']}
orders a week unless stated, "
              f"fixed costs {fixed:,}, owner's pay
{plan['base_owner_pay']:,}.", "",
              "| Scenario | Contribution per order | Surplus before pay |
Surplus after pay | Break-even orders a month |",
```

```
              "|---|---|---|---|---|---|"]
    for r in sensitivity(plan):
        lines.append(f"| {r['scenario']} | {r['contribution']:,.0f} |
{r['surplus_before_pay']:,.0f} | "
                     f"{r['surplus_after_pay']:,.0f} |
{r['break_even_orders_month']} |")
    lines += ["", "## How to use this result", "",
              "FAIL and WARN lines point to claims to verify or fix. Log each
one in your error log "
              "with its source, and re-run after you update the inputs."]
    return "\n".join(lines), counts


def deep_update(base, new):
    for k, v in new.items():
        if isinstance(v, dict) and isinstance(base.get(k), dict):
            deep_update(base[k], v)
        else:
            base[k] = v
    return base


def main(argv=None):
    ap = argparse.ArgumentParser(description="Analyse an AI-drafted business
plan.")
    ap.add_argument("--config", help="JSON file with plan and/or benchmarks
overrides")
    ap.add_argument("--out", help="save the report as Markdown")
    ap.add_argument("--dump-config", help="write the default inputs to a JSON
file and exit")
    args = ap.parse_args(argv)

    plan, bench = copy.deepcopy(DEFAULT_PLAN),
copy.deepcopy(DEFAULT_BENCHMARKS)
    if args.dump_config:
        with open(args.dump_config, "w", encoding="utf-8") as fh:
            json.dump({"plan": plan, "benchmarks": bench}, fh, indent=2,
ensure_ascii=False)
        print(f"Wrote defaults to {args.dump_config}")
        return 0
    if args.config:
        with open(args.config, encoding="utf-8") as fh:
            cfg = json.load(fh)
        deep_update(plan, cfg.get("plan", {}))
        deep_update(bench, cfg.get("benchmarks", {}))
```

```
    report, counts = build_report(plan, bench)
    print(report)
    if args.out:
        with open(args.out, "w", encoding="utf-8") as fh:
            fh.write(report + "\n")
        print(f"\nSaved report to {args.out}")
    return 1 if counts["FAIL"] else 0


if __name__ == "__main__":
    sys.exit(main())
```

Inputs you should replace

- **products:** your own prices, costs and orders per week.
- **claims:** the figures stated in the AI's draft, so the script can check them.
- **benchmarks:** competitor prices, fees and utility costs you have verified yourself. The built-in values are mostly undated listings.
- **uses_gas:** set to true or false so the gas check gives a clear answer.
- **minutes_per_unit and available_hours_per_week:** add these to switch on the capacity check.

Limits

The script tests numbers and assumptions you give it. It cannot prove demand, and its benchmark checks are only as good as the evidence behind them.